

12 — Idempotent Pipelines & Content Hashing

What it is

Idempotent is a word borrowed from math that, in plain language, means: an operation you can safely run over and over on the same input and get the same result each time, without wasted extra work or side effects piling up. Running it once or running it fifty times leaves you in the same place.

The `hash_gate.py` module (R10) is the piece of this project that makes the ETL pipeline idempotent. It computes a short "fingerprint" — an **MD5 hash** — for each category/company record, and compares that fingerprint to the one computed the last time the pipeline ran. If the fingerprint is identical, the record hasn't changed, and the pipeline can safely skip re-translating, re-enriching, and re-embedding it.

An **MD5 hash** is a function that takes *any* input — a sentence, a whole structured record, a big JSON object, doesn't matter how large — and produces a short, fixed-length "fingerprint" string: always exactly 32 hexadecimal characters, regardless of whether the input was 10 bytes or 10 megabytes. Its critical property is: the *same* input always produces the *exact same* fingerprint, and even a *tiny* change to the input (one character, one changed number) produces a *completely different*, unrelated-looking fingerprint.

Real-life analogy

Think of idempotency like a smoke detector's daily self-test: running it once tells you the same thing as running it a hundred times, as long as nothing about the detector has changed — it doesn't drain the battery differently or set the alarm off for real each time. Compare that to a NON-idempotent action like "add \$10 to this bank account" — running that once and running it a hundred times gives you 100 very different balances. Our ETL pipeline needs to behave like the smoke detector, not the bank transfer: re-running it on unchanged data should be a cheap no-op, not something that redoes (and re-charges, and re-risks) all the expensive work again.

The hash itself is like a wax seal on an envelope. You don't need to open two envelopes and compare every word inside letter-by-letter to know if they contain the same letter — you just check whether the seals match. If even one word inside changed, the seal (in this analogy, magically) would look completely different, so a quick glance at the seal tells you everything you need to know.

Why we used it here

Without this, a nightly re-run of the full ETL would blindly re-translate, re-enrich (LLM calls), and re-embed (API calls) **every** record, every single time — even the ones that haven't changed at all since yesterday. Concretely, for this project that's 471 companies and roughly 3,000 categories, each requiring translation, LLM enrichment, and embedding calls per language. Translation, enrichment, and embedding are the slowest stages in the pipeline and the ones most sensitive to free-tier rate limits and quotas (R22 LLM providers, R23 embedding providers). Redoing all of that for records that are byte-for-byte identical to what's already stored in Postgres produces exactly the same output as last time — it's pure waste: slower runs, and quota burned for zero benefit.

The hash gate exists specifically to let the pipeline skip that wasted work: check the cheap fingerprint first, and only pay for translation/enrichment/embedding when the fingerprint proves something actually

changed.

Code walkthrough

From `bilingual_etl/transform/hash_gate.py`:

```
def compute_content_hash(record: dict) -> str:
    canonical = json.dumps(record, sort_keys=True, ensure_ascii=True, default=str)
    return hashlib.md5(canonical.encode("utf-8")).hexdigest()
```

This turns a Python dict into a canonical JSON string, then MD5-hashes that string's bytes into a 32-character hex digest.

The important detail is `sort_keys=True`. Python dictionaries preserve insertion order, but two logically identical records can still be *built* with their keys in a different order — for example, one code path constructs `{"a": 1, "b": 2}` and another constructs `{"b": 2, "a": 1}`. Both represent the exact same data, but `json.dumps` without `sort_keys` would turn them into two *different* text strings (`{"a": 1, "b": 2}` vs `{"b": 2, "a": 1}`), and those different strings would hash to two *different*, wrongly-mismatched fingerprints — the pipeline would incorrectly conclude the record had "changed" when nothing meaningful did. `sort_keys=True` forces the keys into a fixed alphabetical order every time, no matter what order they came in, so the same logical data always produces the same JSON text and therefore the same hash.

This exact scenario is verified by a real passing test,

`tests/etl/test_hash_gate.py::test_key_order_does_not_affect_hash`:

```
def test_key_order_does_not_affect_hash(self):
    record_a = {"a": 1, "b": 2}
    record_b = {"b": 2, "a": 1}
    assert compute_content_hash(record_a) == compute_content_hash(record_b)
```

Next, the decision function:

```
def has_changed(new_hash: str, previous_hash: str | None) -> bool:
    if previous_hash is None:
        return True
    return new_hash != previous_hash
```

Three-way logic, in order:

1. **No previous hash at all** (`previous_hash is None`) — this is the very first time this record has ever been processed, so there is nothing to compare against. Always treat it as changed, because "unchanged" only makes sense relative to something that already exists.
2. **Hashes match** — nothing about the record changed since last time. Safe to skip re-processing.
3. **Hashes differ** — something changed. Must re-process.

Verified with a real, concrete example: a company record with `nr_employees` changing from 13 to 14 flips the hash completely (this is the avalanche effect of a good hash function — a one-character change in the input cascades into a totally different 32-character output), which correctly triggers re-processing via

`test_end_to_end_changed_record`:

```
record = {"company_id": "1", "nr_employees": 10}
```

```
first_hash = compute_content_hash(record)
edited = dict(record)
edited["nr_employees"] = 11
second_hash = compute_content_hash(edited)
assert has_changed(second_hash, first_hash) is True
```

And an identical rerun of the exact same data produces the exact same hash and correctly signals "skip," verified by `test_end_to_end_unchanged_record`:

```
record = {"company_id": "1", "nr_employees": 10}
first_hash = compute_content_hash(record)
second_hash = compute_content_hash(dict(record)) # fresh dict, same data
assert has_changed(second_hash, first_hash) is False
```

Why this module stays "dumb." Notice that `hash_gate.py` never mentions categories, companies, translation, enrichment, or embeddings anywhere. It doesn't know or care *what* it's hashing, and it doesn't decide *what* skipping actually means — skip translation? skip the LLM enrichment call? skip re-embedding? skip the DB write entirely? Those are all different questions with different answers depending on context. That decision belongs to the orchestrator, `bilingual_etl/scripts/main_etl.py` (a later phase of this project), which has the specific context to know it's dealing with a category vs. a company, and which UAT evidence marker applies — `EVIDENCE_AC_HASH_SKIP_CATEGORY` vs. `EVIDENCE_AC_HASH_SKIP_COMPANY`. Keeping `hash_gate.py` generic and reusable — just "does this fingerprint match the last one, yes or no" — means the same small, well-tested utility works for both categories and companies (and anything else added later) without being rewritten or duplicated.

A real schema gap this phase closed. While wiring this up, it turned out the `company_vectors` table was missing a `content_hash` column entirely — only `category_vectors` had one. Since both tables were still completely empty at this point in the project (no data loaded yet), the column was added directly to the schema with zero migration risk — there was no existing data to migrate or backfill. Without that column, hash-based skipping would have been possible for categories but not for companies, which would have silently defeated half the point of this feature.

How to explain this in an interview/to a teammate

"We made the ETL pipeline idempotent by hashing each record's content with MD5 before doing any expensive work on it. If the hash matches what we stored last run, we know nothing changed and skip re-translating, re-enriching, and re-embedding that record — which matters a lot when those steps call rate-limited, free-tier LLM and embedding APIs. The hashing function serializes the record to JSON with `sort_keys=True` first, specifically so that dictionaries with the same data but different key insertion order still produce identical hashes — otherwise you'd get false positives on 'changed' records. The hash-gate module itself is deliberately generic — it just answers 'did this fingerprint change, yes or no' — and has no idea whether it's looking at a category or a company; the orchestrator that calls it decides what 'skip' means in each case. That separation kept a small, easily-tested utility reusable across both entity types instead of duplicating comparison logic in two places."